DATABASES

SQL Joins Visual Guide

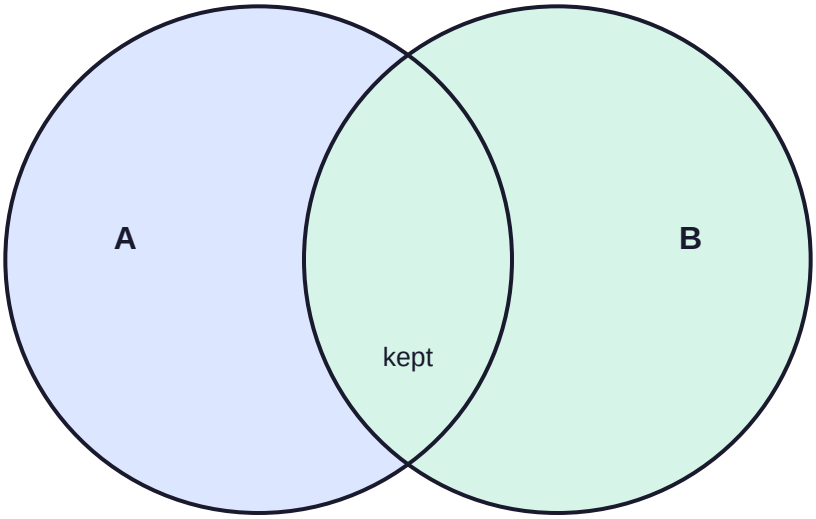
Inner, outer, semi, anti --
and what the engine actually does underneath.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

INNER JOIN: The Overlap

Only rows with a match on BOTH sides survive.



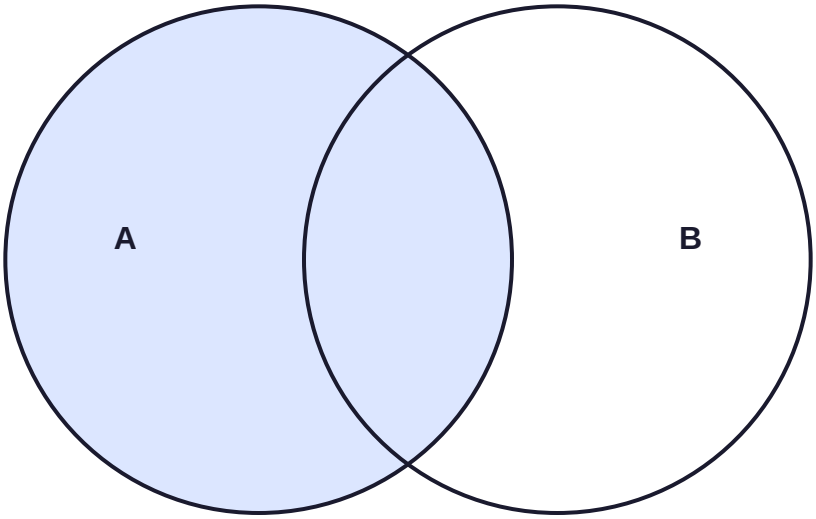

```
SELECT * FROM orders o
INNER JOIN customers c ON o.cust_id = c.id;
-- drops orders with no customer AND customers with no orders
```

The default JOIN. If a side has no match, its rows vanish from the result.

STEP 2/6

LEFT / RIGHT: Keep One Side

Every row from the kept side appears at least once.



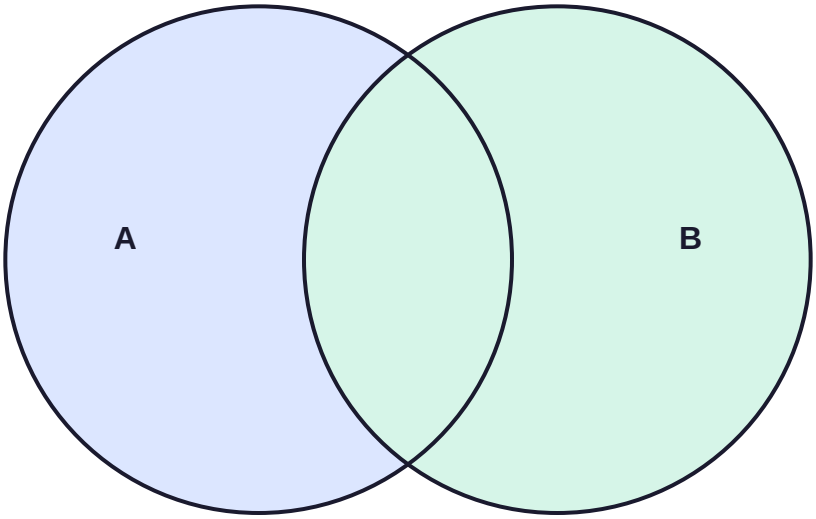

```
SELECT * FROM customers c
LEFT JOIN orders o ON o.cust_id = c.id;
-- all customers; order columns are NULL where no order exists
```

RIGHT JOIN is the mirror image. Most teams standardize on LEFT only -- direction stays readable.

STEP 3/6

FULL OUTER JOIN

Keep EVERYTHING from both sides.




```
SELECT * FROM a FULL OUTER JOIN b ON a.id = b.id;
-- unmatched rows kept, missing side filled with NULLs
```

Rare in app code, common in reconciliation: find rows present in one system but not the other.

STEP 4/6

Semi & Anti Joins

Existence checks -- no row multiplication.

SEMI JOIN
EXISTS / IN

Keep A rows that HAVE a match.

ANTI JOIN
NOT EXISTS

Keep A rows that have NO match.

```
-- semi: customers who placed at least one order
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id);

-- anti: customers with zero orders (NOT IN is NULL-unsafe!)
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.id);
```

Semi/anti joins never duplicate A rows -- unlike a real JOIN, one A row yields at most one output row.

STEP 5/6

Join Algorithms

Three strategies the planner picks between.

Nested Loop

for each A row, scan B -- best when A is tiny / B indexed

Hash Join

hash B into memory, probe with A -- best for big equijoins

Merge Join

both sides sorted, zip together -- best when inputs pre-sorted

EXPLAIN shows the choice. A hash join spilling to disk (work_mem too small) is a classic slowdown.

STEP 6/6

Venn Diagrams, Done Right

What the circles actually mean -- and what they hide.

- The circles are ROW SETS, not tables: each circle = the rows that table contributes.
- The overlap = rows surviving an INNER join. The rest = rows only outer joins keep.
- What Venn hides: DUPLICATES. One A row matching three B rows yields three output rows.
- Venn also hides the join key -- circles say nothing about WHICH columns matched.
- Use Venn for 'which rows survive', then read the ON clause for 'how they matched'.

Mental model: Venn answers survival, the ON clause answers pairing, and cardinality answers row counts.

STEP ✓/✓

Key Takeaways

SQL Joins -- Remember

- INNER keeps only matches; LEFT keeps all of the left side (NULLs fill gaps); FULL OUTER keeps everything.
- Semi (EXISTS) and anti (NOT EXISTS) joins test existence without multiplying rows.
- Engines join via nested loop (small side), hash join (big equijoins), or merge join (sorted inputs).
- Venn shows row survival, not duplicates or join keys -- always check the ON clause.
- Prefer NOT EXISTS over NOT IN: NOT IN breaks silently when the subquery returns NULLs.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev